

NAXML Invoice Template for SSCS (Vendor-Agnostic Format)

Overview: The following template outlines the **NAXML (NACS XML)** invoice format required by SSCS for electronic vendor invoices. It is a vendor-agnostic, invoice-specific XML structure that includes **all required data fields (“column headers”) and metadata** to ensure each item is imported without errors, according to SSCS’s best-in-class specifications. This template contains **no sample values** – it uses placeholders to indicate the data that must be inserted. All examples are provided separately in a step-by-step guide below, keeping the template itself free of mock data as requested.

Invoice File Structure (NAXML ItemSynch)

An SSCS-compatible NAXML invoice file is an XML document with a top-level `<ItemSynch>` element. Within this, it is organized into sections for vendor info, line items, invoice totals, and processing instructions. Every required field is represented by an XML tag (analogous to a “column header” in a spreadsheet). Below is a breakdown of each section and its fields:

Header Section: `<VendorInfo>` (Invoice Metadata)

This section provides identifying information about the vendor and the invoice. It must include:

- **VendorID:** Unique code or ID for the vendor (as recognized in SSCS) ¹ ².
- **VendorName:** Full name of the vendor or supplier ².
- **InvoiceNumber:** The invoice identifier assigned by the vendor (should be unique for each invoice) ².
- **InvoiceDate:** Date of the invoice in `YYYY-MM-DD` format ³.
- **TotalItems:** Total number of line items on the invoice (i.e. the count of `<Item>` entries) ³.
- **StoreLocationID:** Identifier for the store/location receiving the goods (as configured in SSCS).
- **TransmissionDate:** Date the invoice file is transmitted, `YYYY-MM-DD` (often same as or close to InvoiceDate).

Additional metadata fields: Some implementations include fields like **CustomerNumber** (the account number that the vendor assigns to your business), **CurrentCustomerNumber** (if a new/updated account number is in use), or **EDIDeliveryEmail** (the email address where the EDI invoice was sent). These fields are optional and may not be required by all specs, but including them when applicable can provide clarity. For example, the *CustomerNumber* helps identify your account in the vendor’s system. Include these tags if provided by or relevant to the vendor; otherwise, they can be omitted without affecting the import.

Line Items Section: <Items>

Each individual product on the invoice is represented by an <Item> entry containing all relevant item details. **Every <Item> must include the following fields (columns) to ensure a perfect import** ¹:

- **PLU:** The product lookup code or internal item ID. This could be a UPC, SKU, or any item code the system uses to match products. (In many cases, PLU serves as the primary item identifier in the SSCS price book.) ⁴
- **ItemName:** The description or name of the product as it appears on the invoice ⁵.
- **Quantity:** *Optional.* The quantity of this item delivered on the invoice. (In the NAXML ItemSynch format used by some vendors, an explicit <Quantity> field may not appear; instead, quantity is implied by the extended price/cost, as explained below. However, if the schema allows, including a <Quantity> tag is ideal for clarity ⁶.)
- **Price:** The **extended retail price** for the line item – i.e. the total retail value for the quantity delivered (not the unit price) ⁵. This is calculated as the item's retail unit price multiplied by the quantity delivered.
- **Cost:** The **extended wholesale cost** for the line item – i.e. the total cost you (the retailer) pay for that item line ⁵. This equals the vendor's unit cost * times the quantity.
- **Category:** The merchandise category of the item (e.g. *Beverages, Snacks, Wine*). This helps classify the item in the inventory system.
- **Size:** The item's package size or unit measure (e.g. "750ml", "12oz", "1 case"). This is for human reference and item identification.
- **VendorItemCode:** The item's code as used by the vendor. Often this is the same as the PLU or UPC, but it can be a vendor-specific SKU or ordering code. Including it helps cross-reference the vendor's catalog.
- **LastUpdated:** A timestamp indicating when this item entry was last updated, in ISO datetime format (YYYY-MM-DDThh:mm:ssZ). This can be the time of invoice creation or file generation.
- **Status:** The item's status in the system. Typically this is "Active" for items currently carried/delivered. (Other statuses could be "Discontinued" or "Inactive", but normally only active items appear on an invoice.)

Optional item fields:

- **UPC:** The Universal Product Code (12-digit UPC or 13-digit EAN) for the item, if available ⁷. This is crucial for matching if the system relies on UPC. Include the numeric UPC with no spaces.
- **VerifoneUPC:** An 11-digit version of the UPC sometimes used for POS systems that drop the check digit ⁷. (If your POS or SSCS setup requires an 11-digit code, provide it here. Typically this is just the UPC without the last digit.)
- **UPCVerified:** A boolean flag ("true" or "false") indicating if the UPC has been confirmed accurate in the system ⁷. For a new item, or if the UPC is missing, this might be "false". For known items with UPC on file, use "true".
- **UPCNote:** A free-text note used if there's an issue with the UPC. For example, if an item has no UPC in the file, you might include a note like "Manual UPC lookup required." This is informational and helps staff resolve item identity issues.

All required **line-item fields** (PLU, ItemName, Price, Cost, etc.) must be present for each <Item> in order for SSCS to accept the invoice. SSCS will use these to identify the product and the delivered quantity/value ¹. *Notably, the NAXML ItemSynch format does not include a separate quantity field in some vendor*

implementations – instead, the delivered quantity is inferred by comparing the extended cost/price to the known unit cost/price in the price book. If the schema or your implementation allows a `<Quantity>` tag, it's good practice to include it for completeness; otherwise, **ensure that Price and Cost are correctly calculated as extended totals** so the system can deduce the quantity. Every item's data must be accurate and consistent to avoid import errors (e.g., missing UPCs or mismatched costs are common issues to avoid ⁸).

Trailer Section: `<InvoiceTotals>`

After listing all items, the invoice file provides a summary of totals. This section ensures the file's line items add up correctly and serves as a cross-check for the import. The required totals include:

- **SubTotal:** The sum of the extended costs of all items on the invoice (total wholesale amount before any taxes or fees) ⁹. This should equal the sum of all `<Cost>` values from the line items.
- **Tax:** The total tax amount on the invoice ⁹. If the vendor invoice has taxable items, calculate and include the total tax. Use `0.00` if no tax is applied (as is often the case for most merchandise invoices).
- **Total:** The grand total cost of the invoice including tax and any other charges ⁹. In many cases this will equal SubTotal if there are no additional charges or taxes.
- **ItemCount:** The number of line items on the invoice ⁹. This should match **TotalItems** in the header and the count of `<Item>` entries. (It is **not** the sum of quantities of units; it's just the count of distinct line entries.)
- **RetailTotal:** The sum of the extended retail values of all items ⁹. This is the total retail value of the shipment (sum of all `<Price>` fields). It's used for margin and pricing checks, and to alert if there are retail price changes.

Including correct invoice totals is important for validation – the SSCS import process may use these to verify that the line item details were read correctly (e.g. ensuring no line was omitted) and to populate summary records. Always double-check that your SubTotal and RetailTotal match the sum of the item lines, and that Total = SubTotal + Tax (assuming no other fees).

Processing Instructions: `<ProcessingInstructions>`

Finally, the NAXML template ends with a section of processing directives. These tell the SSCS system how to handle the imported data. The key fields here are:

- **ImportType:** Indicates the type of import. For invoice data that includes item information and pricing, the type is usually `"ItemPrice"` (as an Item/Price synchronization file) ¹⁰. This corresponds to the SSCS import format (e.g. NAXML ItemSynch/ItemPrice for vendors like McLane ¹¹ ¹²).
- **UpdateExisting:** Boolean (`true/false`) that tells the system to update existing item records if they already exist in the price book ¹⁰. Typically this is `true` so that any price or cost changes on the invoice will update the item's info in the system.
- **CreateNew:** Boolean that determines if new items not already in the system should be created automatically ¹⁰. Setting this to `true` allows the import to add brand-new items (with the details provided in the invoice) into your inventory database. If `false`, any item on the invoice that isn't recognized will be skipped or flagged (depending on system settings) rather than auto-created. You might use `false` if you prefer to manually review and approve new items.

- **NotifyOnCompletion:** Boolean (`true/false`) indicating if the system should send a notification or alert when the import is completed ¹⁰ . For example, `true` might trigger an email or system message confirming the invoice was processed.
- **UPCCoverage:** A percentage (0–100%) indicating the proportion of line items that had valid UPCs in the file ¹⁰ . This is a metadata statistic – it's not strictly required for processing, but it provides insight into data completeness. SSCS might log or display this to highlight if many items came without UPCs. You should calculate it as
$$\frac{(\text{number of items with UPC})}{(\text{TotalItems})} * 100$$
 . (For example, if out of 10 items, 8 had UPCs, this would be 80.0%.)

The processing instructions ensure the SSCS import routine knows how to treat the incoming data (update vs create items, etc.) and provides helpful metrics. Generally, you'll use the values shown above for a standard invoice import. **Important:** make sure the `<ImportType>` and overall file format match exactly what SSCS expects for that vendor's spec – SSCS supports multiple formats (X12, CSV, PDI, NAXML, etc.), but will only parse the file if it matches the known structure ¹³ ¹⁴ . The template below uses **NAXML ItemSynch/ItemPrice**, which is one of the accepted formats for many convenience-store suppliers ¹¹ ¹² .

Using all the fields above, here is the **complete NAXML invoice template** (in ItemSynch XML format) with placeholder values. **Replace each placeholder with the actual data** for your invoice, and omit any optional elements that you do not have data for. This template is vendor-neutral and can be used for any supplier, as long as you populate it with the correct specifics:

```
<?xml version="1.0" encoding="UTF-8"?>
<ItemSynch version="2.0" timestamp="YYYY-MM-DDThh:mm:ssZ" vendor="VENDOR_CODE">
  <VendorInfo>
    <VendorID>VENDOR_CODE</VendorID>
    <VendorName>VENDOR NAME</VendorName>
    <InvoiceNumber>INVOICE_NUMBER</InvoiceNumber>
    <InvoiceDate>YYYY-MM-DD</InvoiceDate>
    <TotalItems>TOTAL_LINE_ITEMS</TotalItems>
    <StoreLocationID>STORE_ID</StoreLocationID>
    <TransmissionDate>YYYY-MM-DD</TransmissionDate>
    <!-- Optional identifiers, include if applicable: -->
    <!-- <CustomerNumber>ACCOUNT_NUMBER</CustomerNumber> -->
    <!-- <CurrentCustomerNumber>NEW_ACCOUNT_NUMBER</CurrentCustomerNumber> -->
    <!-- <EDIDeliveryEmail>EDI@EMAIL.ADDR</EDIDeliveryEmail> -->
  </VendorInfo>
  <Items>
    <Item>
      <PLU>ITEM_CODE</PLU>
      <ItemName>ITEM DESCRIPTION</ItemName>
      <!-- <Quantity>QTY_DELIVERED</Quantity> (if using an explicit quantity field) -->
      <Price>EXTENDED_RETAIL_PRICE</Price>
      <Cost>EXTENDED_COST</Cost>
      <Category>CATEGORY_NAME</Category>
      <Size>ITEM_SIZE</Size>
```

```

    <VendorItemCode>VENDOR_ITEM_CODE</VendorItemCode>
    <LastUpdated>YYYY-MM-DDThh:mm:ssZ</LastUpdated>
    <Status>Active</Status>
    <UPC>XXXXXXXXXXXX</UPC>
    <!-- If UPC is provided, you can include the 11-digit version: -->
    <VerifoneUPC>XXXXXXXXXXXX</VerifoneUPC>
    <UPCVerified>true</UPCVerified>
    <!-- Include a note if UPC is missing or needs attention: -->
    <!-- <UPCNote>NOTE_TEXT</UPCNote> -->
  </Item>
  <!-- Repeat <Item> block for each product on the invoice -->
</Items>
<InvoiceTotals>
  <SubTotal>TOTAL_EXTENDED_COST</SubTotal>
  <Tax>TOTAL_TAX</Tax>
  <Total>INVOICE_TOTAL_AMOUNT</Total>
  <ItemCount>TOTAL_LINE_ITEMS</ItemCount>
  <RetailTotal>TOTAL_EXTENDED_RETAIL</RetailTotal>
</InvoiceTotals>
<ProcessingInstructions>
  <ImportType>ItemPrice</ImportType>
  <UpdateExisting>true</UpdateExisting>
  <CreateNew>false</CreateNew>
  <NotifyOnCompletion>true</NotifyOnCompletion>
  <UPCCoverage>XX.X%</UPCCoverage>
</ProcessingInstructions>
</ItemSynch>

```

Notes:

- All tag names are **case-sensitive** and must appear exactly as shown (e.g. `<ItemName>` not `<Itemname>`).
- Dates/times should be formatted exactly as specified. The `timestamp` attribute on `<ItemSynch>` and the `<LastUpdated>` fields use ISO 8601 format (the "Z" denotes UTC time) – you can also use your local time with an offset if needed.
- Numeric fields like prices, costs, and totals should include two decimal places and **no currency symbols** (assume currency is USD by default or specify a currency attribute in the XML if the spec supports it) ¹⁵ ¹⁶. Do not use commas in numbers.
- The file should be saved with an `.xml` extension and typically UTF-8 encoding. Ensure it is well-formed XML (all tags closed properly, etc.). Any formatting errors can cause the import to fail.
- Before using this in production, confirm with SSCS support that the vendor's "invoice spec" is indeed NAXML and if there are any vendor-specific quirks. The above template is based on standard NAXML invoice content and SSCS's general requirements ¹¹ ⁶. SSCS can provide spec sheets or example files for reference ¹⁷, but this template should align with the **universal structure** for NAXML invoices accepted by SSCS.

Step-by-Step Guide: Assembling an Invoice File (with Examples)

Now that we have the template structure, this guide will walk through creating a NAXML invoice for a new vendor, using example data to illustrate each step. We will populate the template step-by-step to ensure *every item and field is perfectly formatted* for SSCS import. For this example, let's assume we are setting up an invoice from a vendor **"DABS"** (a fictional code for demonstration) delivering alcoholic beverages – this mirrors a real-world scenario for clarity. **Do not include these example values in your actual template file** – they are for guidance only:

1. **Start the XML and ItemSynch Header:** Begin your file with the XML declaration and the opening `<ItemSynch>` tag. Include the `version`, a `timestamp`, and the `vendor` code. For example:

```
<?xml version="1.0" encoding="UTF-8"?>
<ItemSynch version="2.0" timestamp="2025-08-25T15:12:38Z" vendor="DABS">
```

Here we used `version="2.0"` (the standard NAXML schema version) and a timestamp of August 25, 2025, 15:12:38 UTC. The vendor code is "DABS" for our example supplier ¹⁸. Make sure to replace "DABS" with your vendor's code (and use the correct timestamp) in your actual file.

1. **Fill in `<VendorInfo>` details:** Inside `<ItemSynch>`, add the `<VendorInfo>` section with your vendor and invoice metadata:

```
<VendorInfo>
  <VendorID>DABS</VendorID>
  <VendorName>Utah Division of Alcoholic Beverage Control</VendorName>
  <InvoiceNumber>DABS-INV-1001</InvoiceNumber>
  <InvoiceDate>2025-08-25</InvoiceDate>
  <TotalItems>10</TotalItems>
  <StoreLocationID>HILLS_HOLLOWS_BOULDER</StoreLocationID>
  <TransmissionDate>2025-08-25</TransmissionDate>
</VendorInfo>
```

In this example, **VendorID** is `DABS` and **VendorName** is the full name of that vendor ². **InvoiceNumber** is set to `DABS-INV-1001` – you would use the actual invoice number from the vendor (ensure it matches exactly, including any prefixes or leading zeros the vendor uses). **InvoiceDate** is the date on the invoice. **TotalItems** is 10, meaning we expect 10 line items. **StoreLocationID** is `HILLS_HOLLOWS_BOULDER` (a fictional store code) – use the identifier that SSCS has for your store or site. **TransmissionDate** in this case is the same day, indicating the file was sent on the invoice date. If your vendor provides an account number for your business, you could also include `<CustomerNumber>` here (not shown above). For instance, if DABS assigned account 6242 to us, we could add `<CustomerNumber>6242</CustomerNumber>` ¹⁹. This isn't strictly required but is useful for reference.

1. **List each `<Item>` with all required fields:** Now begin the `<Items>` section and add each product from the invoice as an `<Item>` entry. For each line item, populate the fields as per the template:

```

<Items>
  <Item>
    <PLU>039593</PLU>
    <ItemName>SUGAR HOUSE VODKA 1750ml</ItemName>
    <!-- Quantity is not explicitly included in this format -->
    <Price>227.94</Price>
    <Cost>182.35</Cost>
    <Category>SPIRITS</Category>
    <Size>1750ml</Size>
    <VendorItemCode>039593</VendorItemCode>
    <LastUpdated>2025-08-25T15:12:38Z</LastUpdated>
    <Status>Active</Status>
    <UPC>615260026006</UPC>
    <VerifoneUPC>61526002600</VerifoneUPC>
    <UPCVerified>true</UPCVerified>
  </Item>
  <Item>
    <PLU>087123</PLU>
    <ItemName>ARETTE CLASICA BLANCO TEQUILA 1000ml</ItemName>
    <Price>395.88</Price>
    <Cost>316.70</Cost>
    <Category>SPIRITS</Category>
    <Size>1000ml</Size>
    <VendorItemCode>087123</VendorItemCode>
    <LastUpdated>2025-08-25T15:12:38Z</LastUpdated>
    <Status>Active</Status>
    <UPC></UPC>
    <UPCVerified>false</UPCVerified>
    <UPCNote>Manual UPC lookup required for this item</UPCNote>
  </Item>
  <!-- (Add additional <Item> blocks for each product, up to TotalItems count)
-->
</Items>

```

In the first `<Item>` above, we've provided a detailed example ²⁰ ⁷: The product code **PLU** is `039593`, and **VendorItemCode** is also `039593` (in this case the vendor's code matches the PLU). **ItemName** is "SUGAR HOUSE VODKA 1750ml" which clearly identifies the item and size. **Category** is "SPIRITS" (a predefined category in our system for liquor) and **Size** is "1750ml". **Status** is "Active". We included the **UPC** `615260026006` (the 12-digit UPC from the bottle) and **VerifoneUPC** `61526002600` (the first 11 digits) ⁷, marking **UPCVerified** as true since we trust this UPC. The **Price** is `227.94` and **Cost** is `182.35`. These represent the *extended totals* for this line. In other words, if the vendor delivered, say, 6 bottles of this vodka, the wholesale cost per bottle might be \$30.39 and the retail price \$37.99 each – multiplying by 6 yields the extended cost \$182.35 and extended retail \$227.94. **Notice:** there is no separate `<Quantity>` field – the quantity (6 units in this hypothetical) is inferred by those totals and the known unit price/cost. This is why it's crucial that the Price and Cost here are the **totals** for that line item ²¹. The SSCS import will divide the extended Cost by the item's cost in the price book to figure out how many units to add into

inventory. If your format supports an explicit quantity, you would include it (e.g., `<Quantity>6</Quantity>`), but in this ItemSynch example it's omitted.

In the second `<Item>` example, we show a case where the item (Arette Tequila) had **no UPC available**. The `<UPC>` tag is left empty (`<UPC/>`) and we set `UPCVerified` to false ²². We also included a `<UPCNote>` to flag that a manual UPC lookup is required. Providing such notes can help your staff follow up on items missing barcodes. The Price for this tequila is `395.88` and Cost `316.70`, which could correspond to, for instance, 12 bottles at \ \$26.39 cost each, retailing at \ \$32.99 each – again the extended totals are used.

Continue adding an `<Item>` block for every product on the invoice. Ensure each has all the required tags (PLU, ItemName, Price, Cost, etc.). Double-check the values: **Price = unit retail * quantity**, **Cost = unit cost * quantity** for each line. Also verify text fields for typos (especially ItemName and codes). By the end of this step, you will have as many `<Item>` entries as the number you put in TotalItems (in our case, 10 items).

1. **Close the Items section and add Invoice Totals:** After listing all items, close the `</Items>` tag and append the `<InvoiceTotals>` section. Here you'll sum up the invoice's values:

```
<InvoiceTotals>
  <SubTotal>2006.42</SubTotal>
  <Tax>0.00</Tax>
  <Total>2006.42</Total>
  <ItemCount>10</ItemCount>
  <RetailTotal>2508.06</RetailTotal>
</InvoiceTotals>
```

In our example, the **SubTotal** (total wholesale cost) is \ \$2006.42 and since this invoice had no tax, **Tax** is 0.00, making **Total** \ \$2006.42 as well ⁹. **ItemCount** is 10, matching the number of `<Item>` lines (and the *TotalItems* we set in VendorInfo). The **RetailTotal** (total retail value) is \ \$2508.06 ⁹. These totals should be computed from the line items we entered: if you add up all the `<Cost>` values from the items, the sum should equal 2006.42; likewise, sum of all `<Price>` values should equal 2508.06. It's good practice to double-check these calculations. Any discrepancy might indicate a typo in an item's price or cost. SSCS will likely check that ItemCount and totals align with the items – mismatches could trigger an import error or at least a warning. For instance, if you accidentally put ItemCount 9 while actually listing 10 items, the system would know something is off. Ensuring consistency here contributes to a "perfect" invoice import with no errors.

1. **Specify Processing Instructions:** Lastly, include the `<ProcessingInstructions>` block to instruct SSCS how to handle the imported data, then close the `<ItemSynch>` tag:

```
<ProcessingInstructions>
  <ImportType>ItemPrice</ImportType>
  <UpdateExisting>true</UpdateExisting>
  <CreateNew>false</CreateNew>
  <NotifyOnCompletion>true</NotifyOnCompletion>
```



```
<UPCCoverage>90.0%</UPCCoverage>
</ProcessingInstructions>
</ItemSynch>
```

For this example, **ImportType** is `ItemPrice`, telling SSCS we're sending an item-level invoice/price update file ¹². **UpdateExisting** is true – so if any of these 10 items already exist in our database, their info (cost, price, etc.) will be updated with the invoice data. **CreateNew** is false, which means if an item isn't recognized (e.g., a brand new product not in our system), the file won't automatically create it – instead, it might be flagged for manual review. (If you prefer new items to be auto-added, you'd set this to true. Just ensure you have all necessary fields like UPC, Category, etc., present to create a complete new item record.) **NotifyOnCompletion** is true to get a notification once the import finishes – this could be an email or just a log entry depending on configuration. **UPCCoverage** is set to 90.0% in our example, because out of 10 items we had 9 with UPCs (one item was missing a UPC) ¹⁰. We calculated that as $9/10 * 100\%$. In your case, compute this percentage based on how many items in your invoice have valid UPC values. While **UPCCoverage** is not a functional necessity, it provides insight; SSCS may log it or use it to prompt you if coverage is low (e.g. "Only 50% of items had UPCs – consider updating your database"). If unsure or if not using this field, you can omit it, but since our goal is a best-in-class import, we include it here for completeness.

1. **Final file checks:** Ensure that every opened tag has a corresponding closing tag. The structure should exactly match the template hierarchy: `<ItemSynch>` wrapping everything, inside it `<VendorInfo>` (closed), `<Items>` (with multiple `<Item>` children, all properly closed), then `<InvoiceTotals>`, then `<ProcessingInstructions>`, and finally the closing `</ItemSynch>`. The order of sections should be exactly as above. Also verify that numeric fields use a dot as decimal separator and have two decimals (even if .00). Remove any placeholder comments or example lines you had in place – the final file should only contain the XML with your actual data.
2. **Save and name the file appropriately:** Typically, the file can be named anything `.xml`, but sometimes vendors/SSCS have conventions. For example, you might name it `INV_1001_DABS.xml` or something descriptive. Make sure it has the `.xml` extension. If emailing this to SSCS (or if the vendor emails it), the SSCS system will pick it up from the designated EDI inbox ²³. Double-check the email or transfer method per your SSCS setup (in our case, we'd ensure the vendor sends the file to our `v6242s1@edidelivery.com` address).
3. **Test the import (if possible):** It's wise to do a trial run. Send the file to the SSCS system (or use any test import function if available) and see if it imports without errors. If everything is formatted correctly, SSCS will create a pending delivery record with all 10 items, with the costs, quantities, and prices populated as provided ²⁴ ²⁵. You can then review in CDB Win -> Vendor Import or Direct Store Deliveries module, verify the data, and post the invoice to update inventory. If there are issues, common ones might be:
 4. A required field was missing or misspelled (e.g., forgot `<VendorID>` or used a wrong tag name).
 5. The file isn't well-formed XML (missing a closing tag, etc.).
 6. Data type mismatch (letters in a numeric field, etc.).
 7. The vendor code or item codes don't match what the system expects (in our example, SSCS must have vendor "DABS" set up and the items either existing or allowed to be created).

If any errors occur, fix the file accordingly and re-test. Once the file imports cleanly, you have your “perfect” template that can be reused for future invoices – you’ll just plug in new values each time.

By following this template and guide, you ensure **every required column and piece of metadata is present** in your NAXML invoice file ¹. This comprehensive approach should result in an error-free import, with all items added to SSCS exactly as on the vendor’s invoice. Always keep the official spec in mind and don’t hesitate to reach out to SSCS support for any clarifications – they can confirm if any vendor-specific tweaks are needed ¹⁷. With the above structure, however, you’re equipped with a universally compatible NAXML invoice template ready to integrate new vendors into your SSCS system seamlessly.

Sources:

- SSCS EDI setup correspondence and documentation ²⁶ ¹³ ¹
- Example NAXML invoice content (ItemSynch format) from SSCS integration tests ² ²⁰ ⁹ ¹⁰
- Connexus (NAXML) standards overview – invoice data requirements ²⁷ (line items with IDs, descriptions, quantity, price, taxes, totals, etc.)
- Petrosoft C-Store Office documentation on NAXML invoice mapping (for field context) ²⁸ ²⁹.

¹ ⁸ ¹¹ ¹² ¹³ ¹⁴ ¹⁷ ²³ ²⁴ ²⁵ ²⁶ Request to confirm EDI Set up format of Vendor DABS for Hills and Hollows, LLC

<https://mail.google.com/mail/u/0/>

² ³ ⁴ ⁵ ⁷ ⁹ ¹⁰ ¹⁸ ¹⁹ ²⁰ ²² SSCS EDI for DABS

<https://docs.google.com/document/d/1AXzdxRw5UaTmUsJYp-0bLtgnvQoVnjbUwHGTUmdBTAc>

⁶ ²¹ ²⁷ Retail Merchandise Data Exchange | Connexus

<https://www.connexus.org/retail-merchandise-data-exchange>

¹⁵ ¹⁶ ²⁸ ²⁹ Store Purchases (Invoice) Connexus Data Mapping

https://help.petrosoftinc.com/Content/Business_Documents/store_purchases_connexus_data_mapping.htm?TocPath=Products%7C%7C%7CBusiness%20Documents%20API%20Sandbox%7C____8